Decoding Bayes' Theorem: The 4 Pillars Behind Every Prediction

In Part 1, we saw *why* Naive Bayes is fast. Now let's open the hood and see how Reverend Thomas Bayes' 18th-century formula turns raw feature counts into a classification decision (GeeksforGeeks).

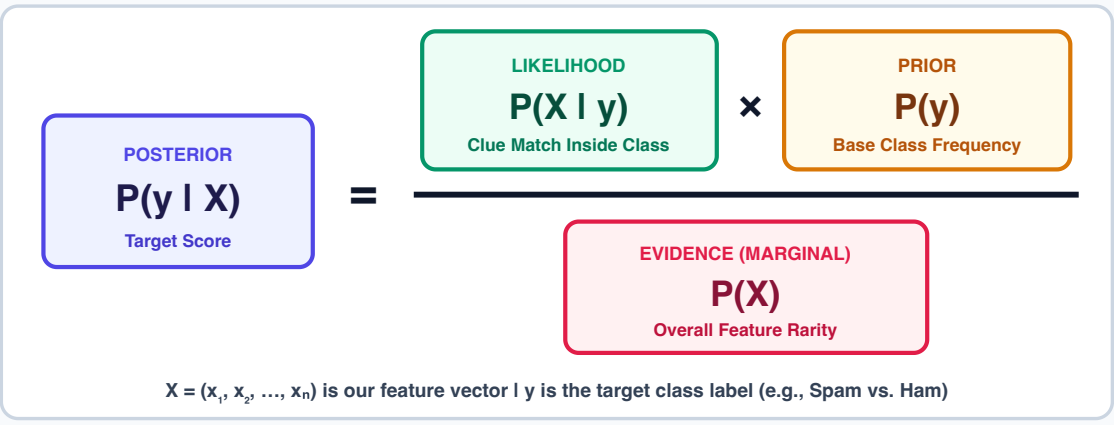


Figure 2.1: Every Naive Bayes classifier combines Prior knowledge with Feature Likelihoods to compute Posterior probability.

Term & Symbol	Plain-English Meaning	Spam Filter Intuition
Posterior — P(y | X)	Probability of class **y** *after* seeing feature vector **X**.	*"Given this email says 'Free Lottery', what is the chance it is Spam?"*
Likelihood — P(X | y)	Probability of observing features **X** inside class **y**.	*"Out of all known Spam emails, how often do the words 'Free Lottery' appear?"*
Prior — P(y)	Baseline frequency of class **y** before looking at **X**.	*"What percentage of all emails in my inbox are Spam overall?"*
Evidence — P(X)	Total probability of features **X** across all classes.	*"How common is the phrase 'Free Lottery' across every email ever sent?"*

1. Why Do We Flip P(y | X) Into P(X | y)?

When a new patient walks into a clinic or a new email hits your inbox, you want **P(Disease | Symptoms)**. Trying to look up that exact combination directly fails whenever a symptom combination is rare—so Bayes' Theorem flips the question around!

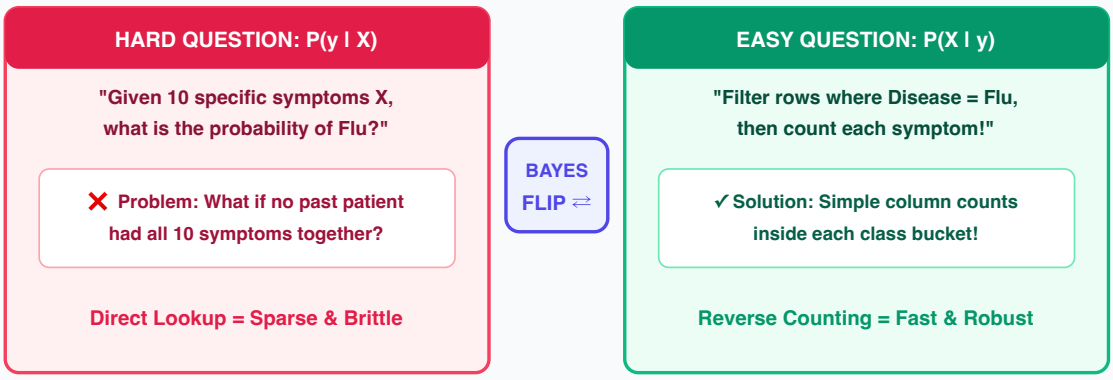


Figure 2.2: Bayes' Theorem converts a hard prediction question into simple column counting insi each class group.

Step	How Naive Bayes Uses the Flip on Your Training Table	SQL / Pandas Equivalent
1. Split by Class	Group all training rows by target label **y** (e.g., separate *Play = Yes* rows from *Play = No* rows).	`df.groupby('y')`
2. Compute Prior P(y)	Count how many rows belong to class **y** divided by total rows **N**.	`len(df_y) / len(df)`
3. Compute $P(x_i \mid y)$	Inside each class group, count how often feature value x_i appears.	`df_y['x_i'].value_counts(normalize=True)`

2. Unpacking the Feature Vector $X = (x_1, x_2, \ldots, x_n)$

Real datasets don't have just one clue—they have a whole row of features $X = (x_1, x_2, \ldots, x_n)$. Let's trace how the **Naive Independence Assumption** simplifies the numerator (GeeksforGeeks):

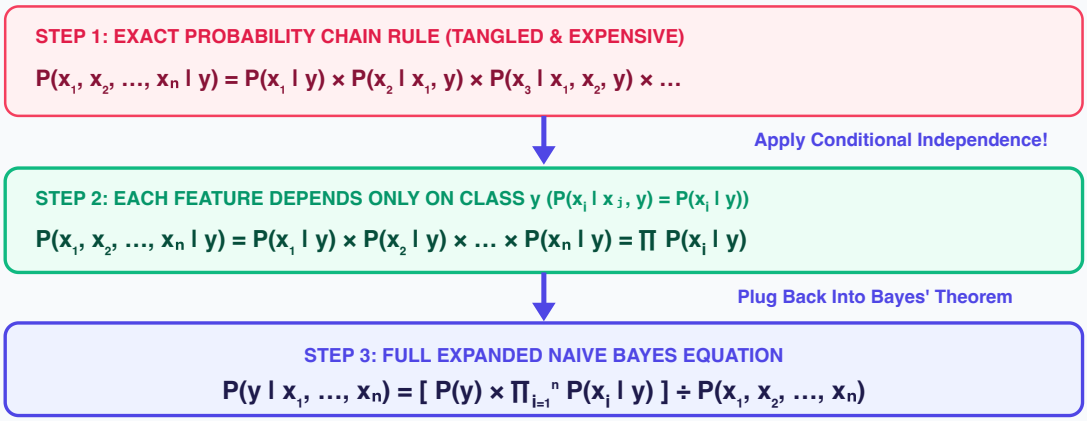


Figure 2.3: Conditional independence collapses a nested chain of conditional probabilities into clean product $\prod P(x_i \mid y)$.

Mathematical Symbol	What It Represents	Concrete Example (Golf Dataset)
$X = (x_1, x_2, x_3, x_4)$	The complete row of **n** observed features for a test sample.	**X** = (*Sunny, Hot, Normal, False*)
$\prod_{i=1}^{n} P(x_i \mid y)$	Capital Pi ($\prod$) means **multiply** all individual feature likelihoods together.	P(*Sunny*|Yes) × P(*Hot*|Yes) × P(*Normal*|Yes) × P(*False*|Yes)

3. The Shortcut: Why We Drop the Denominator P(X)

When comparing **Class = YES** vs. **Class = NO** for the exact same input row **X**, notice that the denominator **P(X)** never changes! Because dividing two candidates by the same constant doesn't change who wins, we drop **P(X)** during classification (GeeksforGeeks).

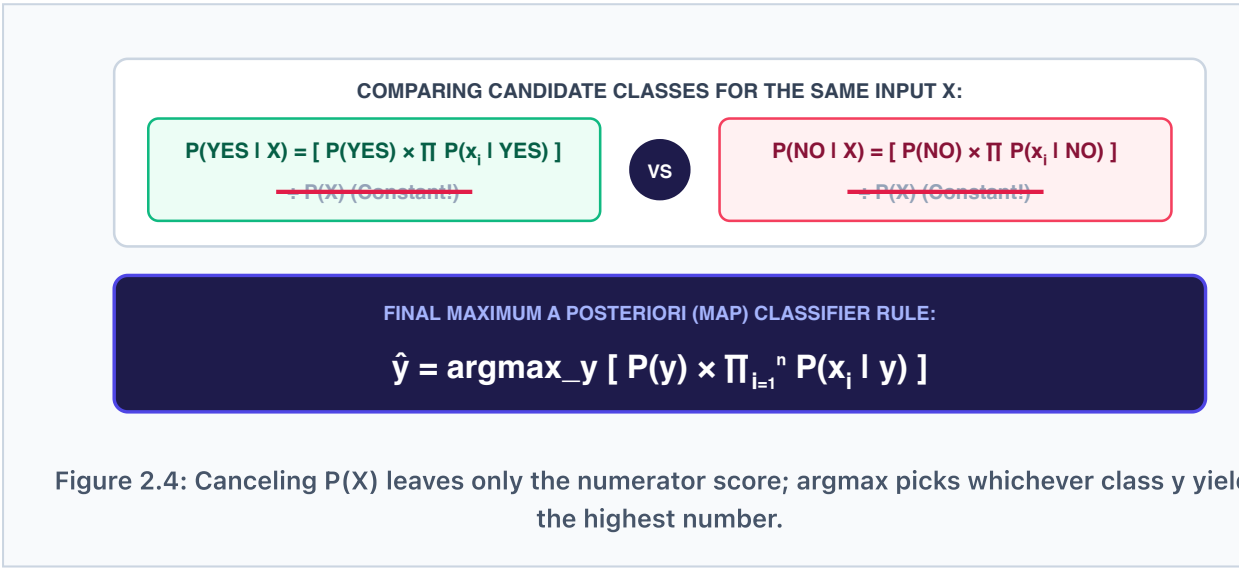


Figure 2.4: Canceling P(X) leaves only the numerator score; argmax picks whichever class y yiel the highest number.

Wait—When Do We Still Need P(X)? If you only want the winning label **ŷ**, you never need **P(X)**. If you want clean percentages that add up to **100%**, simply divide each class's numerator score by the **sum of all class numerator scores**!

Step in Normalization	Formula	Example Score Conversion
1. Raw Numerator Scores	Score(y) = P(y) × ∏ P(x_i | y)	Score(YES) = **0.0141** | Score(NO) = **0.0068**
2. Normalized Percentage	P(YES | X) = Score(YES) ÷ [Score(YES) + Score(NO)]	0.0141 ÷ (0.0141 + 0.0068) = **67.5% YES** vs. **32.5% NO**

4. Production Secret: Why Libraries Add Logs Instead of Multiplying

What happens when an email has **500 words** and you multiply 500 small probabilities like **0.01 × 0.01 × ... × 0.01**? In 64-bit floating-point math, $0.01^{500} = 10^{-1000}$ rounds straight down to **0.0** (*Numerical Underflow*)!

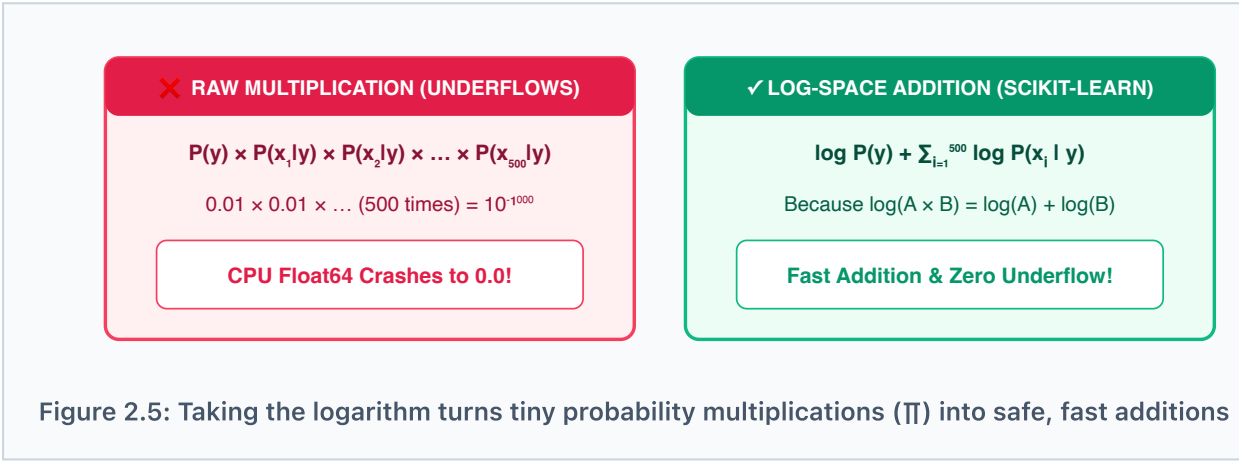


Figure 2.5: Taking the logarithm turns tiny probability multiplications (∏) into safe, fast additions

Part 2 Master Cheat Sheet	Equation / Rule to Remember
1. Bayes' Theorem	$P(y \mid X) = [\, P(X \mid y) \times P(y) \,] \div P(X)$
2. Naive Independence Expansion	$P(X \mid y) = P(x_1 \mid y) \times P(x_2 \mid y) \times \ldots \times P(x_n \mid y) = \prod_{i=1}^{n} P(x_i \mid y)$
3. MAP Decision Rule	$\hat{y} = \text{argmax}_y \, [\, P(y) \times \prod_{i=1}^{n} P(x_i \mid y) \,]$ (Denominator P(X) drops out)
4. Log-Space Production Rule	$\hat{y} = \text{argmax}_y \, [\, \log P(y) + \sum_{i=1}^{n} \log P(x_i \mid y) \,]$ (Prevents float underflow)

Reference & Coming Up Next

1. GeeksforGeeks, "Naive Bayes Classifiers — Mathematical Formulation," Machine Learning Tutorial: https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/ — **Coming in Part 3:** Full Hand-Calculated Walkthrough on the 14-Row Golf Dataset!